Program

The purpose of this program is to determine whether a 9 × 9 Sudoku board is a valid solution using multithreading. The program reads the Sudoku board from a file named input.txt and checks whether each row, column, and 3 × 3 subgrid contains the numbers 1 through 9 exactly once. The program uses the POSIX Threads (Pthreads) library to perform multiple validation tasks concurrently. It supports two execution modes, selected through a command-line argument. Option 1 uses one thread to check all nine columns, while Option 2 creates nine separate threads to check the columns individually. Both options also create nine threads to check the rows and nine threads to check the subgrids. After all threads finish executing, the program combines their results to determine whether the board is a valid Sudoku solution. It also measures and displays the execution time.

Input Processing

The read_board() function opens input.txt and reads 81 integers into a two-dimensional array named board[9][9]. Each integer represents one cell of the Sudoku board. The function verifies that all 81 integers can be read and rejects the input if additional integers are present. If the file cannot be opened or does not contain exactly 81 integers, the program terminates with an error code. Before starting the validation process, the program prints the board to the console so the user can see the input being checked.

Checking Sudoku Values

The valid_values() function determines whether a group of nine integers contains every number from 1 through 9 exactly once.It uses an array named seen to track which numbers have already appeared. If a number is outside the valid range or has already been encountered, the function returns 0, indicating that the group is invalid. If all nine numbers are valid and unique, the function returns 1. This function is used by the worker threads to validate the rows, columns, and subgrids.

Thread Management

The program uses the pthread.h library to create and manage threads. Each thread executes the check_worker() function, which determines which portion of the Sudoku board it must validate. The ThreadData structure stores three pieces of information:

- kind: Identifies the type of validation the thread performs.
- index: Specifies the row, column, or subgrid to check.
- result_index: Identifies where the thread stores its validation result.

The program uses a pthread_t array to store thread identifiers and a ThreadData array to store the information passed to each thread. Each worker thread validates its assigned section independently and writes the result to the corresponding position in the shared results array. Because each thread writes to a separate array element, the program avoids conflicts between threads when recording validation results.

Option 1: Single Thread for All Columns

When the program is executed with the argument 1, it creates 19 threads:

- 9 threads to validate the rows.
- 1 thread to validate all nine columns.
- 9 threads to validate the 3 × 3 subgrids.

The column-validation thread checks each column sequentially. It extracts the nine values in a column, passes them to valid_values(), and then proceeds to the next column if the current column is valid. This option uses fewer threads because one thread performs all column checks.

Option 2: Separate Thread for Each Column

When the program is executed with the argument 2, it creates 27 threads:

- 9 threads to validate the rows.
- 9 threads to validate the columns individually.
- 9 threads to validate the 3 × 3 subgrids.

Each column has its own thread, allowing the column checks to run independently of one another. This provides more opportunities for parallel execution because the program does not require a single thread to validate all nine columns sequentially. Option 2 creates more threads than Option 1, but this does not automatically mean it will execute faster. Thread creation and scheduling introduce overhead, and the Sudoku validation tasks are relatively small.

Synchronization and Result Collection

After creating the required threads, the main function calls pthread_join() for every thread. This ensures that the main thread waits until all validation tasks have completed before evaluating the results. The program then examines the results array. If every validation task succeeds, it prints SOLUTION: YES. If any row, column, or subgrid is invalid, it prints SOLUTION: NO. The program also uses the C clock() function to measure CPU time during thread creation, execution, and joining. The elapsed time is printed alongside the final result. The program creates multiple POSIX threads that can execute independently. On a system with multiple available CPU cores, the operating system can schedule these threads to run simultaneously on different cores. Option 1 creates 19 threads, while Option 2 creates 27 threads. In both cases, row, column, and subgrid checks are independent, so the operating system has opportunities to distribute the work across multiple cores. However, creating multiple threads is not the same as guaranteeing that multiple cores are used. The operating system controls thread scheduling, and the actual degree of parallel execution depends on the number of available CPU cores, system load, thread scheduling, and other factors.

Evidence Supporting the Answer

The source code provides evidence that the program is capable of parallel execution:

1. **Use of POSIX threads:** The program includes pthread.h and uses pthread_create() to create multiple independent worker threads.
2. **Independent validation tasks:** Each worker thread checks a separate row, column, or subgrid without needing the other threads to finish first.
3. **Separate result locations:** Each thread stores its result in a different element of the results array, allowing the checks to proceed independently.
4. **Thread joining:** The calls to pthread_join() ensure that all worker threads finish before the main thread evaluates the results.

These details demonstrate that the program is designed for concurrent execution and can use multiple CPU cores.

This program demonstrates how multithreading can be applied to Sudoku validation. It divides the board into independent validation tasks and uses POSIX threads to check rows, columns, and subgrids concurrently. Option 1 uses 19 threads and assigns all column checks to a single thread, while Option 2 uses 27 threads and assigns each column to its own thread. Option 2 provides more opportunities for parallel execution, although the additional thread overhead may offset any performance improvement for such a small workload. The program is capable of using multiple CPU cores because its independent worker threads can be scheduled simultaneously by the operating system. Nevertheless, the source code alone cannot confirm actual multicore utilization. Performance-monitoring evidence or per-thread CPU scheduling measurements are needed to demonstrate that multiple cores were used during a specific run.

Experiment

The objective of this experiment is to determine whether there is a statistically significant difference in execution time between two methods of validating a Sudoku board using POSIX threads. The experiment compares Option 1, which uses one thread to validate all nine columns, with Option 2, which creates a separate thread for each column. The experiment uses 50 timing measurements for each method. The recorded execution times are used to calculate the average, measure variability, and perform a statistical hypothesis test.

The null hypothesis and alternative hypothesis are defined as follows:

Null hypothesis (H_0): There is no statistically significant difference between the average execution times of the two methods.
H_0:mu_1=mu_2

Alternative hypothesis (H_1): There is a statistically significant difference between the average execution times of the two methods.
H_1:mu_1≠mu_2

Here, (mu_1) represents the population mean execution time for Option 1, and (mu_2) represents the population mean execution time for Option 2.The significance level is set to (α=0.05). If the p-value is

less than 0.05, the null hypothesis will be rejected. Otherwise, there will not be sufficient evidence to reject it.

The experiment uses the execution-time measurements recorded in the spreadsheet(fig1).

Option 1	Option 2		Averages	
0.0009	0.0022		Option 1	0.000938
0.001	0.0011		Option 2	0.001256
0.0009	0.0007			
0.0008	0.0014			
0.0007	0.001			
0.0006	0.0008			
0.0007	0.0009			
0.0007	0.0011			
0.0009	0.0007			
0.0006	0.0008			
0.0013	0.0011			
0.0013	0.0011			
0.0008	0.001			
0.0008	0.0008			
0.0008	0.001			
0.0008	0.0013			
0.0008	0.001			
0.001	0.0012			
0.0013	0.0013			
0.0008	0.0026			
0.0008	0.0022			
0.0008	0.0011			
0.0013	0.0018			
0.0012	0.002			
0.0011	0.0015			
0.0013	0.0013			
0.001	0.0019			
0.0011	0.0008			
0.001	0.0009			
0.0012	0.0007			
0.0013	0.0009			
0.0013	0.0009			
0.0008	0.0011			
0.0011	0.001			
0.0011	0.001			
0.0013	0.0009			
0.001	0.0008			
0.001	0.0007			
0.0007	0.0009			
0.0008	0.0013			
0.0008	0.0011			
0.0008	0.0008			
0.0009	0.001			
0.0008	0.0013			
0.0009	0.001			
0.0008	0.0011			
0.0008	0.0011			
0.0008	0.0008			
0.0008	0.007			
0.0008	0.0008			

fig1

Option 1 creates 19 threads: nine for rows, one for all columns, and nine for subgrids. Option 2 creates 27 threads: nine for rows, nine for individual columns, and nine for subgrids.

The two methods are evaluated using the same type of Sudoku validation task. The recorded execution times are treated as two samples, with 50 observations per method.

The following statistics are calculated:

- Mean execution time for each method.
- Standard deviation to measure the variability of the execution times.
- Welch's independent two-sample t-test to compare the mean execution times without assuming equal population variances.
- A 95% confidence interval for the difference between the mean execution times.

Welch's t-test is appropriate for this comparison because the two methods have different observed variances. The test uses a two-sided alternative because either method could be faster.

Results

The following results were calculated from the recorded timing measurements.

Statistic	Option 1	Option 2
Number of runs	50	50
Mean execution time	0.000938 s	0.001256 s
Median execution time	0.000850 s	0.001000 s
Standard deviation	0.000209 s	0.000930 s

Option 1 has a lower mean execution time than Option 2. The average for Option 1 is approximately 0.000938 seconds, while the average for Option 2 is approximately 0.001256 seconds. The difference between the mean execution times is:0.000318 Based on these measurements, Option 2 takes approximately 33.9% longer on average than Option 1. The standard deviation for Option 2 is also substantially larger, indicating greater variability in its recorded execution times. Welch's two-sample t-test was used to determine whether the observed difference in mean execution time is statistically significant. The test produced the following results:

Test statistic	Result
t-statistic	-2.360
Degrees of freedom	53.94
p-value	0.0219

Significance level	0.05
95% confidence interval for Option 2 minus Option 1	0.000048 to 0.000588 seconds

The p-value of approximately 0.0219 is less than the significance level of 0.05. Therefore, the null hypothesis is rejected. The experiment provides statistically significant evidence of a difference in average execution time between the two methods. The 95% confidence interval for the difference in means is entirely positive. This supports the conclusion that Option 2 takes longer on average than Option 1 under the conditions represented by these measurements.

Conclusion

This experiment compared two multithreaded methods of validating a Sudoku board using 50 recorded execution times per method. The null hypothesis stated that there was no statistically significant difference in the average execution times.

The calculated mean execution times were 0.000938 seconds for Option 1 and 0.001256 seconds for Option 2. Welch's t-test produced a p-value of approximately 0.0219, which is below the 0.05 significance level. Consequently, the null hypothesis was rejected.

The experimental results indicate that Option 1 was statistically faster than Option 2 for the recorded runs. This suggests that creating additional threads does not necessarily improve performance when the tasks are small and thread-management overhead is significant.